Algorithms in computing solutions

01-E01 Ordering, preservation, and stability

The first candidate preserves all four records and has ordered keys 1,1,2,2. It violates stability because a precedes c in the input but follows c in the output. The second candidate loses identity d; its ordered keys 1,2,2 do not repair the failed permutation condition. A fully correct output is [(1,b),(1,d),(2,a),(2,c)], preserving b before d and a before c. Empty input returns []. Removing duplicate-key records is invalid unless the specification explicitly changes.

01-E02 First match and termination

Searching for 8 compares $8 = 8$ at index 1 and returns 1. Searching for 9 compares $8 \neq 9, 3 \neq 9, 8 \neq 9, 5 \neq 9$, then reaches $i = 5$. The bound test fails, so $A[5]$ is not read and NIL is returned.

At each loop entry, all elements in $A[1..i-1]$ differ from v. This holds for the empty prefix and remains true because advancement follows a failed equality test. The variant $n - i + 1$ is nonnegative before the body and decreases by 1, precluding infinite iteration. The invariant proves minimality of the returned index; the variant proves termination. Short-circuit evaluation avoids reading $A[n+1]$. At $n = 0$, $1 \leq 0$ is false, giving NIL with zero key comparisons.

01-E03 Partial correctness is insufficient

Only the even branch terminates, and it returns the required 0. Every terminating execution is therefore correct. The legal input $n = 1$ never terminates, violating the universal termination condition. The replacement `return 0` terminates in finitely many steps with the required result on every nonnegative input. Passing all even-input tests cannot exclude an odd-input counterexample, and the proof may not silently restrict the legal domain.

01-E04 Paths and tours

The direct edge 02 costs 2, as does 012. Every edge costs at least 1, and the direct edge costs 2, so no 0-to-2 path costs less than 2. Fixing 0 leaves six directed orders of the other vertices. Reversal pairs have equal cost in this undirected graph, leaving

Algorithms in computing solutions (continued)

Tour	Cost
01230	$1+1+1+1=4$
01320	$1+2+1+2=6$
02130	$2+1+2+1=6$

All candidates have been considered, proving optimum 4 for this instance. Paths do not require all vertices or a return to the start; tours do. General tour counts grow factorially, so this finite enumeration does not establish polynomial time for arbitrary n .

01-E05 Conditions for randomized majority

Majority error requires at least two errors. Its probability is monotone in each independent error rate, so its maximum under rates at most $1/4$ occurs when all three equal $1/4$. Thus

$$3(1/4)^2(3/4)+(1/4)^3=10/64=5/32.$$

Equivalently, independent uniform-variable coupling dominates each error indicator by an independent Bernoulli$(1/4)$ indicator. Success is at least $27/32$, and serial time is at most 3 s. Error remains possible. Identical shared random choices can make errors perfectly correlated, invalidating products. Three different path outputs may all be correct, so they need a checker or selection rule rather than binary majority semantics.

01-E06 Integer threshold in one model

The inequality becomes $g(n)=n-20\lg n>0$. Since $g'(x)=1-20/(x\ln 2)$, g decreases on $[2,20/\ln 2]$ and then increases. The decreasing interval has no positive value because $g(2)=-18$. The increasing interval has only one crossing. Numerically,

$$\lg 143\approx 7.1598713,\quad g(143)\approx -.1974267,$$
$$\lg 144\approx 7.1699250,\quad g(144)\approx .6015000.$$

Hence $N=144$. At 143, B still costs more; monotonicity prevents later reversal. Extending the model to 1 gives $B(1)=0<A(1)=2$, followed by reversal at 2. An isolated solution is not a persistent threshold. Real one-element algorithms have base costs, so the modeled zero is not a measured zero time.

Algorithms in computing solutions (continued)

01-E07 Input bits and hardware units

At 1024, A uses $2 \cdot 1048576 = 2097152$ steps and takes .02097152 s. B uses $40 \cdot 1024 \cdot 10 = 409600$ steps and takes .4096 s. A's 100-fold hardware advantage wins at this input size. An L-bit positive integer is at most $2^L - 1$, so the loop has worst-case $\Theta(2^L)$ iterations. The value 31 is binary 11111, has length 5, and causes 31 iterations. Fewer operations do not necessarily mean fewer seconds on different machines. $\Theta(N)$ in numerical value is exponential in encoded length.

01-E08 Parallelism is not a processor cutoff

Work is $W = 6 + 3 = 9$; span is $S = 1 + 3 = 4$. The independent phase requires exactly $\lceil 6/P \rceil$ slots, and the dependent chain requires three more, so $T_P = \lceil 6/P \rceil + 3$. For $P = 1, 2, 3, 6$, times are 9,6,5,4 and speedups are 1,1.5,1.8,2.25. Average parallelism is $W/S = 9/4 = 2.25$. Although $3 > 2.25$, increasing from 3 to 6 processors improves time from 5 to 4. The ratio bounds maximum speedup, not the processor count beyond which benefits stop. All times satisfy $T_P \geq \max(9/P, 4)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Algorithms in computing solutions (continued)

Verification code appendix: algorithm_contracts.py

```
"""Finite checks for original chapter examples, not universal proofs."""
from itertools import product,permutations
from fractions import Fraction as F
from math import log2,comb,ceil
from collections import Counter

def first_match(a,v):
    i=0
    while i<len(a) and a[i]!=v:i+=1
    return i+1 if i<len(a) else None

def sort_contract(source,result,stable=False):
    if Counter(source)!=Counter(result):return False
    if any(result[i][0]>result[i+1][0] for i in range(len(result)-1)):return False
    if stable:
        for key in {x[0] for x in source}:
            if [x[1] for x in source if x[0]==key]!=[x[1] for x in result if
x[0]==key]:return False
    return True

if __name__=='__main__':
    count=0
    for n in range(6):
        for a in product(range(3),repeat=n):
            for v in range(3):
                expected=next((i+1 for i,x in enumerate(a) if x==v),None)
                assert first_match(a,v)==expected;count+=1
    src=[(2,'a'),(1,'b'),(2,'c'),(1,'d')]
    assert sort_contract(src,sorted(src,key=lambda r:r[0]),True)
    assert not sort_contract(src,[(1,'b'),(1,'d'),(2,'c'),(2,'a')],True)
    assert not sort_contract(src,[(1,'b'),(2,'a'),(2,'c')])
    def w(a,b):return 1 if (a-b)%4 in (1,3) else 2
    tours=[]
    for p in permutations((1,2,3)):
        if p[0]<p[-1]:
            q=(0,)+p+(0,);tours.append((q,sum(w(a,b) for a,b in zip(q,q[1:]))))
    assert sorted(v for q,v in tours)==[4,6,6]
    assert sum(F(comb(3,k))*F(1,4)**k*F(3,4)**(3-k) for k in (2,3))==F(5,32)
    assert 143-20*log2(143)<0<144-20*log2(144)
    assert [ceil(6/p)+3 for p in (1,2,3,6)]==[9,6,5,4]
    print('FIRST-MATCH finite cases:',count,'; tour costs:',tours)
    print('crossover g143/g144:',143-20*log2(143),144-20*log2(144),'majority error<=5/32')
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study